SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	N.Hema Sree	**Reg. No.:**	192511100
Date:	6/10/2026	**Duration:**	60 minutes

Code of Conduct

I N.Hema Sree (192511100) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: N.Hema Sree

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement** ○ Explain the problem in your own words. ○ Identify the objectives of the proposed system.
    - ○ State the scope of the problem.
2. **Identify Key Stakeholders and Challenges** ○ Identify all the stakeholders involved in the system. ○ Explain the roles and responsibilities of each stakeholder.
    - ○ Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach** ○ Describe the existing system or current process. ○ Identify its strengths and limitations.
    - ○ Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles** ○ Design a software-based solution for the given problem.

- Describe the major functional modules of the proposed system. ○ Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. **Justify the Chosen Methodology/Framework**
   - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps). ○ Justify why the selected methodology/framework is suitable for the proposed system.
   - Explain how it supports successful project development and maintenance.
6. **Discuss Expected Outcomes and Limitations** ○ Describe the expected benefits of the proposed solution. ○ Explain how the solution addresses the identified challenges. ○ Discuss potential risks, implementation challenges, and limitations. ○ Suggest possible future enhancements.

28. Secure API Marketplace and Third-Party Integration Governance Platform

1. Understanding the Problem Statement

1.1 Introduction and Industry Context

Organizations increasingly depend on third-party APIs for payments, maps, communication, artificial intelligence, analytics, authentication, and other digital services. These APIs accelerate software development but also introduce security, reliability, compatibility, privacy, and dependency risks. A systematic governance platform is therefore required to manage external APIs throughout their lifecycle.

1.2 Problem in Our Own Words

Organizations often manage third-party APIs using scattered documentation, spreadsheets, emails, and individual development practices. This makes it difficult to know which APIs are being used, whether they are secure, which versions are active, who approved them, and whether they remain reliable. The proposed platform provides a centralized way to discover, assess, approve, monitor, version, and retire third-party APIs.

1.3 Objectives of the Proposed System

• Maintain a centralized inventory of third-party APIs.

• Assess API security and operational risks.

• Track API versions and compatibility.

• Provide API documentation and discovery.

• Manage approval and access policies.

• Securely manage API credentials.

• Monitor API availability, latency, and performance.

• Maintain audit trails for API usage and approvals.

• Integrate API validation and security checks into CI/CD pipelines.

• Support controlled API retirement and replacement.

1.4 Scope of the Problem

In Scope	Out of Scope
API discovery, registration and documentation	Development of third-party APIs
Security/risk assessment and approval	Replacement of all existing enterprise applications
Credential and access management	Physical security of provider infrastructure
Monitoring, audit and usage tracking	Formal legal certification
Versioning, CI/CD validation and retirement	Unrelated business application development

2. Key Stakeholders and Challenges

2.1 Identification of Stakeholders

The major stakeholders are application developers, API administrators, security teams, DevOps/CI-CD engineers, business or product owners, compliance and audit teams, third-party API providers, and system administrators.

2.2 Roles and Responsibilities

Stakeholder	Role and Responsibilities
Application Developers	Discover approved APIs, integrate APIs, test compatibility and report issues.
API Administrators	Register APIs, maintain documentation, versions and approval workflows.
Security Team	Perform security assessments, vulnerability checks and risk classification.
DevOps Engineers	Integrate API validation and security checks into CI/CD pipelines.
Business Owners	Define requirements, evaluate business value and approve integrations.
Compliance/Auditors	Review policies, access records and audit trails.
API Providers	Maintain API availability, documentation, security and version information.

System Administrators	Maintain infrastructure, monitoring, backups and availability.

2.3 Major Challenges

• Lack of centralized information about third-party APIs.

• Difficulty identifying insecure or unreliable APIs.

• Unauthorized API usage and inconsistent approval.

• Poor management of API keys and credentials.

• Breaking changes between API versions.

• Limited visibility into API performance and availability.

• Difficulty tracking API ownership and approvals.

• Security vulnerabilities in third-party integrations.

• Dependency on external API providers.

• Difficulty retiring outdated APIs safely.

3. Analysis of the Current Approach

3.1 Existing System / Process

In a conventional approach, developers independently search for third-party APIs and integrate them into applications. Documentation may remain on provider websites, while approval information, credentials, and security reviews are handled through separate emails, documents, or spreadsheets. Monitoring is often performed separately by individual teams. Version or availability problems may be discovered only after an application fails.

3.2 Strengths of the Current Approach

• Easy for developers to start using APIs.

• Minimal initial setup.

• Developers can select APIs based on immediate requirements.

• Existing provider documentation and tools can be used directly.

• Suitable for small projects with limited integrations.

3.3 Limitations of the Current Approach

• No centralized API inventory.

• Inconsistent security assessment.

• Manual approval processes.

• Poor visibility of API dependencies.

• Credentials may be stored insecurely.

• Difficult API version tracking.

• Limited auditability.

• Delayed detection of API failures.

• High risk of unauthorized integrations.

• Difficult API retirement and replacement.

3.4 Gap Analysis

Aspect	Current Approach	Proposed Platform
Discovery	Manual search	Central API marketplace
Security	Manual/inconsistent	Automated security and risk assessment
Approval	Email/manual	Workflow-based approval
Versioning	Developer dependent	Version tracking and alerts
Credentials	Scattered storage	Secure credential management
Monitoring	Separate tools	Central monitoring dashboard
Audit	Partial records	Central audit trail
CI/CD	Manual validation	Automated pipeline checks

Retirement	Ad-hoc	Controlled retirement workflow

4. Proposed Solution Using Software Engineering Principles

4.1 Solution Overview

The proposed system is a centralized, cloud-based Secure API Marketplace and Third-Party Integration Governance Platform. It acts as a controlled gateway between application teams and external API providers. Developers can search the marketplace, view documentation, review security and risk information, and request access. Security and business teams can review and approve integrations before production use. The platform continuously monitors approved APIs and generates alerts for security vulnerabilities, performance degradation, version changes, and availability problems.

4.2 Proposed Architecture

The architecture can be organized into five layers: Presentation Layer for developer, administrator and security dashboards; API Gateway and Security Layer for authentication, authorization, rate limiting and validation; Governance and Application Layer for catalog, approval, risk, version and monitoring services; Data Layer for API registry, audit records, dependencies and encrypted credentials; and Infrastructure Layer for cloud resources, containers, logging, monitoring and backups.

4.3 Major Functional Modules

Module	Description	Key Functions
M1 API Marketplace	Central API catalog	Search, categories, details, documentation
M2 API Registration	Registers external APIs	Provider, endpoints, versions, ownership
M3 Security & Risk	Evaluates API risks	Vulnerability checks, risk scoring, review
M4 Approval & Governance	Controls approval	Policies, review, approval/rejection
M5 Credential Management	Protects access	API keys, OAuth tokens, secret management
M6 Version & Dependency	Tracks changes	Compatibility, version alerts, dependencies
M7 Monitoring	Monitors API health	Availability, latency, errors, alerts
M8 Audit & Reporting	Maintains records	Usage logs, approvals, reports
M9 CI/CD Integration	Validates integrations	Security and policy checks
M10 Retirement	Manages lifecycle	Deprecation, migration, retirement

4.4 Software Engineering Principles

• Modularity: Functions are separated into independent modules.

• Scalability: Services can be scaled independently as usage increases.

• Maintainability: Versioned APIs and well-defined interfaces make changes easier.

• Security: Authentication, authorization, encryption and secure credential storage protect integrations.

• Reliability: Health monitoring, alerts and fallback mechanisms reduce service disruption.

• Usability: Developers can discover and understand APIs through one marketplace.

• Interoperability: REST-based interfaces support existing enterprise applications.

• Auditability: Registration, approval and access activities are recorded.

4.5 API Integration Workflow

• Developer searches for an API in the marketplace.

• Documentation, provider information and risk score are reviewed.

• Integration request is submitted.

• Security team performs assessment.

• Business owner reviews the requirement.

• Approval policies are checked.

• Credentials are securely provisioned.

• API is integrated into the application.

• CI/CD pipeline performs automated validation.

• Production usage is continuously monitored.
• Version or security alerts are generated.
• Deprecated APIs are migrated and safely retired.

4.6 Functional Requirements

• Users shall authenticate securely and sessions shall be protected.
• Administrators shall register and update third-party API information.
• Developers shall search and filter APIs in the marketplace.
• The system shall maintain API documentation and version information.
• The system shall perform security and risk assessments.
• API access shall require appropriate authorization.
• Credentials shall be stored securely.
• The system shall provide an approval workflow.
• The system shall monitor API availability and performance.
• The system shall generate alerts for failures, vulnerabilities and version changes.
• Governance activities shall be recorded in an audit log.
• The platform shall integrate security validation with CI/CD pipelines.
• The system shall track API dependencies.
• Administrators shall manage API versions and deprecation.
• The system shall support controlled API retirement and replacement.

4.7 Non-Functional Requirements

Category	Requirement
Performance	Marketplace and governance operations should respond quickly under normal load.
Scalability	Support increasing APIs, users and applications.
Availability	Monitoring and failover mechanisms should provide high availability.
Security	Encryption, authentication, authorization and secure secret management.
Reliability	Health checks, monitoring and recovery mechanisms.
Usability	Simple search, documentation and approval interfaces.
Maintainability	Modular services and documented APIs.
Auditability	Complete and tamper-resistant governance records.
Interoperability	REST/JSON and standard API protocols.
Portability	Containerized deployment across cloud environments.

4.8 Security and Governance Controls

• Role-based access control for developers, administrators, security teams and auditors.
• Encryption of API credentials and sensitive configuration.
• OAuth 2.0 or equivalent token-based authorization.
• TLS for data transmitted between services.
• Rate limiting and request validation at the API gateway.
• Automated vulnerability and dependency scanning.
• Approval policies before production access.
• Audit logs for registration, approval, access and retirement.
• Continuous monitoring and anomaly alerts.

5. Justification of the Chosen Methodology / Framework

5.1 Candidate Models Considered

Criterion	Waterfall	Spiral	Agile (Scrum)	DevOps

Changing requirements	Low	Medium	High	High
Stakeholder feedback	Low	Medium	High	High
Risk management	Low	High	High	High
Continuous security	Low	Medium	High	High
Frequent releases	Low	Medium	High	High
Long-term maintenance	Medium	Medium	High	High

5.2 Selected Methodology: Agile Scrum with DevSecOps

The recommended methodology is Agile Scrum supported by DevSecOps practices. The platform can be developed through short iterations or sprints. API discovery, security assessment, approval, monitoring and CI/CD integration can be developed incrementally. Security testing and governance checks are included throughout the development lifecycle rather than being performed only at the end.

5.3 Why This Methodology is Suitable

• Changing requirements: API providers, security threats and organizational policies change frequently.

• Continuous feedback: Developers, security teams and business owners can review every sprint.

• Early risk detection: Security and integration problems can be identified early.

• Continuous security: Automated security checks can be included in CI/CD pipelines.

• Incremental delivery: Marketplace, assessment, monitoring and retirement modules can be delivered step by step.

• Better maintenance: New API standards, versions and security requirements can be added in future sprints.

5.4 Proposed Release Plan

Release	Sprints	Major Deliverables
R1 – Foundation	1–2	Architecture, authentication, marketplace and registration
R2 – Governance	3–4	Security assessment, approval workflow and policies
R3 – Integration	5–6	Credential management, version tracking and CI/CD integration
R4 – Monitoring	7–8	Monitoring, alerts, audit reports and dependency tracking
R5 – Lifecycle	9–10	Deprecation, migration, retirement and final security testing

6. Expected Outcomes and Limitations

6.1 Expected Benefits

• Improved visibility of all third-party API dependencies.

• Reduced security and operational risks.

• Faster and more controlled API discovery.

• Better management of API credentials.

• Improved API reliability and performance.

• Early detection of vulnerabilities and failures.

• Reduced unexpected compatibility problems.

• Stronger governance and auditability.

• Safer CI/CD integration.

• Easier API migration, replacement and retirement.

• Scalable and maintainable integration management.

6.2 How the Solution Addresses the Challenges

Challenge	Proposed Solution
APIs are difficult to discover	Central API marketplace
Security risks are unclear	Security and risk assessment module
Manual approval	Automated governance workflow

Credential leakage	Secure credential management
Version conflicts	Version and dependency management
API failures	Continuous monitoring and alerts
Poor accountability	Central audit and reporting
Unsafe deployments	CI/CD security validation
Outdated APIs	Deprecation and retirement management

6.3 Potential Risks and Implementation Challenges

Risk	Likelihood	Impact	Mitigation
Third-party API failure	Medium	High	Monitoring, fallback and alternative providers
Credential leakage	Low	High	Encryption, secret vault and access control
API version incompatibility	Medium	High	Version tracking and compatibility testing
Security vulnerability	Medium	High	Automated scans and periodic assessment
Integration complexity	Medium	Medium	Standard APIs, SDKs and adapter layer
False security alerts	Medium	Medium	Risk scoring and manual review
Provider changes	Medium	Medium	Version monitoring and notifications
Adoption resistance	Medium	Medium	Simple UI, training and gradual migration

6.4 Limitations of the Proposed Solution

The proposed platform still depends on the security and reliability of external API providers. Automated security assessment cannot identify every possible vulnerability. Continuous monitoring and governance also increase infrastructure and operational costs. Some legacy applications may require additional adapters before they can integrate with the platform. The system can reduce third-party integration risks, but it cannot completely eliminate failures caused by external providers, network problems, regulatory changes, or previously unknown vulnerabilities.

6.5 Suggested Future Enhancements

• AI-based API risk prediction.

• Automated API security testing.

• AI-assisted API documentation generation.

• Intelligent recommendation of alternative APIs.

• Predictive detection of API failures.

• Automated migration between API versions.

• Advanced dependency visualization.

• Zero-trust API access.

• Immutable governance records.

• Automated compliance reporting.

7. Conclusion

The Secure API Marketplace and Third-Party Integration Governance Platform provides a centralized approach for managing the complete lifecycle of external API integrations. It improves API discovery, security assessment, approval, credential management, monitoring, version control, auditing and retirement. Agile Scrum with DevSecOps enables continuous development while keeping security and governance integrated into every stage. The proposed solution therefore provides organizations with a secure, scalable, maintainable and well-governed approach to third-party API integration.

A Scrum-based lifecycle with DevSecOps practices was selected because it accommodates changing regulations, validates the privacy-utility trade-off early, embeds security in every sprint and supports long-term maintenance. Although limitations such as possible accuracy loss and added operational overhead remain, the expected outcomes are higher user trust, reduced data-exposure risk and a scalable platform that supports responsible use of user data. With the future enhancements outlined, the platform can evolve into a durable foundation for ethical, intelligent digital services.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system,	Good analysis with most strengths and	Limited analysis with partial	Proposed solution is unclear or inappropriate;
	strengths, weaknesses, and identifies all gaps.	weaknesses identified.	identification of issues.	requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.

Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25